

371 Mini Project 2 - Connection Oriented TCP-like protocol

Paul Pham - Khang Phan

Introduction:

This report will specify our connection oriented and reliable protocol via detailed specification on how it works, code explanation, test suite explanation, manual tests results, testing across the network (two different laptops with two different IPs) results and fairness proof.

This will present the protocol in the best way and prove how it achieved connection oriented, reliable, congestion control, flow control and performance.

Inside the zip, you'll find the pcap files of: performance test run where it captured our testing run using our own automated test suite to test the performance of the protocol, sender and receiver side over the network test (where we set up two different laptops and tested the protocol across the network). Detailed explanation on the automated test suite and the across the network test are below in the test procedures section. We also included an excel file that holds the manual observation of our protocol runs and a graph on how the cwnd and rwnd behaves. Again, details on this will be in the report.

I. Protocol specifications:

1. Socket Type:

The protocol is implemented over the UDP socket type (SOCK_DGRAM). Since UDP does not provide reliability, ordering, flow control, or congestion control, hence all of these mechanisms are implemented in sender.py and receiver.py.

2. Packet format:

Each packet has the following fields (Packet.py)

Field	Description
seq	Sequence Number
ack	Acknowledgement Number
rwnd	Receiver window size (for Flow Control)
checksum	8-bit check
flags	SYNC, ACK, FIN
payload	Data Bytes

3. Corruption + ACK/NAK

Check Sum mechanism:

- Each packet includes a checksum field computed from the payload, sequence number, ACK number, and rwnd
- Checksum computation: $(\text{sum}(\text{payload}) + \text{seq} + \text{ack} + \text{rwnd}) \% 256$
- Both sender and receiver verify packet integrity by recomputing and comparing checksums

Corruption handling:

- Sender side: Corrupted ACKs are discarded and ignored
- Receiver side: Corrupted data packets trigger retransmission of the last valid ACK
- No explicit NAK packets - duplicate ACKs serve as implicit NAK

4. Sequence numbers

Purpose

- Uniquely identify each packet in transmission
- Detect duplicates and out-of-order packets
- Enable proper ordering at the receiver side

Implementation:

- Sequence numbers start at 0 and increment by 1 for each new packet
- Sender maintains nextseqnum (next packet to send) and `base` (oldest unacknowledged packet)
- Receiver maintains expectedseqnum (next expected in-order packet)

Detailed explanation:

Our protocol uses a per-connection sequence number field in every data packet header. The sender initializes $\text{seq} = 0$ at the start of a connection and increments the sequence number by 1 for each new data segment sent. The receiver tracks expectedseqnum , the next in-order sequence number it expects.

ACK packets use the ack field to carry the highest in-order sequence number received so far, supporting cumulative acknowledgments.

5. Duplicated ACK only

Purpose:

- Detect packet loss without waiting for timeout
- Enable fast retransmit mechanism (TCP Reno feature)

Implementation:

- Sender tracks the last ACK number received (`last\_ack\_received`)
- Counter (`dup\_ack\_count`) increments when same ACK is received multiple times
- Three duplicate ACKs trigger fast retransmit

Detailed explanation:

The protocol uses only positive acknowledgments (ACKs); there are no dedicated NAK packets. When the receiver successfully gets an in-order, uncorrupted segment with sequence number `expectedseqnum`, it delivers the payload to the application, increments `expectedseqnum`, and sends an ACK with `ack = last_in_order_seq`, while corrupted or out-of-order packets cause it to resend the last ACK as an implicit NAK. On the sender side, duplicate ACKs are counted, and when 3 duplicate ACKs for the same sequence number are observed, the sender performs a fast retransmit of the presumed lost segment and adjusts the congestion window accordingly.

Benefits:

- Faster loss recovery than timeout-based retransmission
- Maintains higher throughput during sporadic losses
- Part of TCP Reno congestion control algorithm (fast retransmit)

6. Loss detection + timeouts + reliability**Implementation:****1. Timeout based (primary):**

- Timer starts when first packet in window is sent
- If ACK not received within `TIMEOUT\_INTERVAL` (1.0s), packet is considered lost
- Triggers retransmission of entire unacknowledged window

2. Duplicate ACKs based (secondary):

- Three duplicate ACKs indicate packet loss
- Triggers fast retransmit of specific lost packet

Reliability guarantees:

- All packets are eventually delivered in order
- Lost packets are retransmitted (via timeout or fast retransmit)
- Corrupted packets are detected and discarded, triggering retransmission
- Duplicate packets are detected and dropped by receiver

Corruption handling:

- Sender: Corrupted ACKs are discarded and ignored (triggering eventual timeout)
- Receiver: Corrupted data packets are discarded and trigger duplicate ACKs for the last in-order segment

Unreliable simulations:

- Data packets are randomly dropped with probability `LOSS_PROB = 0.3` (30%)
- Control packets (SYN, SYN-ACK, ACK, FIN, FIN-ACK) are never intentionally dropped
- This simulates realistic network loss while ensuring connection management reliability

Detailed explanation:

The underlying UDP channel is unreliable, so our protocol provides reliability in software using simulated loss, checksums, ACKs, and timeouts. Data packets are randomly dropped with probability `LOSS_PROB` in `udt_send`, while control packets (e.g., SYN/FIN) are never intentionally dropped. Each packet carries a checksum over header and payload; corrupted packets are discarded and trigger duplicate ACKs for the last in-order segment. The sender starts a retransmission timer for the oldest unacknowledged packet and, on timeout, retransmits all unacknowledged packets from base. Together, cumulative ACKs, duplicate ACKs, and timeout-based retransmissions ensure that every segment is eventually delivered correctly and in order despite simulated loss and corruption.

7. GBN like Pipelining + Cumulative ACKs**Pipelining:**

- Multiple packets can be in-flight simultaneously
- Window size controlled by minimum of:
 - Pipeline limit (configured window size)
 - Receiver window (flow control)

- Congestion window (congestion control)

Cumulative ACKs:

- ACK(n) acknowledges all packets up to and including sequence number n
- Receiver only sends ACK for the last in-order packet received
- Out-of-order packets are dropped and trigger duplicate ACK

Detailed explanation:

The sender implements a Go-Back-N-style sliding window with cumulative ACKs, maintaining base (oldest unacknowledged sequence number) and nextseqnum (next number to send) so that all packets with seq in [base, base + window_size) may be in flight at once. The receiver tracks a single expectedseqnum and only accepts the next in-order, uncorrupted packet; any packet with seq > expectedseqnum or a corrupted packet is discarded, and the receiver re-sends an ACK for expectedseqnum - 1 as a duplicate ACK. Each ACK is cumulative and confirms receipt of all packets up to and including ack. When the sender receives an ACK with ack ≥ base, it advances base to ack + 1, slides the window, and stops or restarts the timer as needed, allowing pipelining with simple receiver logic.

8. Flow control

Purpose:

- Prevent sender from overwhelming receiver's buffer
- Receiver advertises available buffer space in each ACK

Mechanism:

- Receiver maintains buffer_capacity (8 packets) and buffer_occupancy
- Available space computed as: $rwnd = buffer_capacity - buffer_occupancy$
- Sender respects receiver's advertised window

Detailed explanation:

Flow control is implemented using a receiver-advertised window (rwnd). The receiver maintains a fixed buffer_capacity and a dynamic buffer_occupancy, and for each ACK it advertises the available space available = buffer_capacity - buffer_occupancy in the rwnd field. The sender computes an effective send window as $effective_win = \min(GBN_window, receiver_rwnd, cwnd)$ and does not send new data when

$\text{nextseqnum} - \text{base} \geq \text{effective_win}$, ensuring that the receiver's buffer is never overrun.

9. Congestion Control

Algorithm:

TCP Reno-style with Slow Start, Congestion Avoidance, and Fast Recovery

Components:

1. Congestion Window (cwnd):

- Dynamic window size based on network conditions
- Starts at 1 packet (slow start)

2. Slow start threshold (ssthresh)

- Initially set to 8 packets
- Dividing line between slow start and congestion avoidance

Detailed explanation:

Our congestion control uses a congestion window cwnd and a slow-start threshold ssthresh . The sender starts in slow start with $\text{cwnd} = 1$ and increases cwnd by 1 segment per RTT (per new ACK) until $\text{cwnd} \geq \text{ssthresh}$, then switches to congestion avoidance, where cwnd grows roughly by $1/\text{cwnd}$ per ACK (linear growth). On a timeout, the sender sets $\text{ssthresh} = \text{cwnd} / 2$, resets $\text{cwnd} = 1$, and retransmits all unacknowledged packets. If it receives 3 duplicate ACKs for the same ack, it infers a lost segment, performs a fast retransmit, and reduces cwnd /updates ssthresh , allowing the protocol to probe available bandwidth while backing off under congestion.

10.3 way handshake

Purpose:

- Establish reliable connection before data transfer
- Synchronize sequence numbers between sender and receiver
- Verify both parties are ready for communication

Timeout handling:

- Both sides implement retransmission with 5-second timeout

- Prevents indefinite blocking on lost handshake packets
- Ensures eventual connection establishment

Detailed explanation:

The protocol is connection-oriented and uses a three-way handshake similar to TCP to establish state before data transfer. The sender, starting in CLOSED, calls `connect()`, sends a SYN packet, and enters SYN_SENT; the receiver, in LISTEN, receives SYN, records the sender's address, replies with SYN-ACK, and moves to SYN_RCVD. When the sender receives SYN-ACK, it sends a final ACK and enters ESTABLISHED, and the receiver transitions to ESTABLISHED upon receiving this ACK. Both sides use timeouts and possible retransmissions during the handshake for robustness, and connection teardown is handled with a separate FIN/ACK four-way close

11. Code explanation (methods inside the class):

Reason why we write receiver and sender instead of client and server?

We implement a receiver and sender because we are implementing the transport layer instead of the application layer. These roles focus on packet sequencing, reliability, flow control, and congestion control, and not on providing or requesting services. Using "sender" and "receiver" better reflects the behavior of a transport protocol, where one endpoint transmits data and the other acknowledges it.

Receiver class:

receiver.py

__init__

- Initializes receiver state, buffer, and expected sequence number.

accept()

- Handles the 3-way handshake: wait SYN → send SYN-ACK → wait ACK → connected.

udt_send()

- Sends a raw encoded packet over the UDP socket.

udt_rcv()

- Receives a raw packet from UDP and decodes it; returns None if nothing arrives.

rdt_rcv()

Main receive logic:

- Simulated loss
- Corruption check
- In-order / out-of-order handling
- Sends appropriate ACKs
- Handles FIN shutdown.

deliver_data()

- Prints delivered payload (simulating handing data to the application).

Sender class:

sender.py

__init__

Initializes sender state: windows, cwnd/sssthresh, timers, congestion vars.

connect()

- Performs sender side of handshake: send SYN → wait SYN-ACK → send ACK.

start_timer()

- Starts retransmission timer.

stop_timer()

- Stop timer.

timer_expired()

- Returns True if timeout interval passed.

check_events()

- Checks for incoming ACKs and triggers timeout if needed.

udt_send()

- Sends encoded packets through UDP.

udt_rcv()

- Receives raw packet; returns decoded packet or None.

rdt_send(data)

Core send logic:

- Wait for available window
- Build packet, send, start timer if needed
- Process ACKs immediately.

_process_ack(pkt)

- Handles ACKs, duplicate ACKs, updates base, adjusts congestion window.

_timeout_resend()

- Handles timeout: reset cwnd, set ssthresh, and resend all unacked packets.

close()

- Performs FIN handshake to close connection cleanly.

II. Testing procedures and result:

As part of the test, we've created a test suite that can simulate the real connection environment, calculating and monitoring everything from reliability, throughput, flow control, congestion control and connection management of the protocol. These are un-biased black box test suites where it uses the Sender class and simply calls its methods to communicate with the Receiver class, mimicking actual application using an underlying connection oriented protocol. The code for the test suite will be found on our Github in the resources

and references part of the report as `prove_performance.py` Python file. However, the code snippet and output of the test results are included in this report via screenshots. We also included Wireshark's proof as supplementary as well.

How to run our automated testing suite:

The main test suite you can run is `prove_performance.py`. Open two separate terminals, run `prove_performance.py` on one, and `receiver_test.py` on the other one. The `prove_performance.py` script will act as the sender and `receiver_test.py` will be the receiver. Both of these scripts interact with the base protocol code on the Sender and Receiver class. Prove performance script runs through 5 main proof or tests, Reliability, Throughput, Congestion Control, Flow Control and Connection Management. For the reliability test, we intentionally drop 30% of the packets and observe how the protocol behaves, see if it retransmits the packets correctly and all packets are delivered reliably at the end despite loss. For throughput, we tested by sending 100 packets to the receiver, keeping track of a timer and the size of all the packets and calculating the throughput in kb/s. For Congestion Control, we keep track of the values evolution of `cwnd`, see how it behaves with the network traffic. For flow control, we test if the sender acknowledges the receiver broadcast buffer window and respects it. Lastly, for connection management, we verify the 3-way handshake initially and the tear down after the connection finishes.

This suite contains our automated test. In our results, the protocol passed all these tests successfully. Please try it out and see the logs and observe how the protocol behaves under the automated test environment. Another test suite we have is `test_retransmit` which simulate packet loss and the packet is automatically retransmitted and can be observed through Wireshark.

For the report, we have included manual tests via Wireshark and terminal observation as well, which is the section below.

1. Connection oriented

A) Connection establishment (handshake)

Three-way handshake establishes the connection before data transfer.

Sequence:

Frame 1: [Sender:62319 → Receiver:5000] SYN

Frame 2: [Receiver:5000 → Sender:62319] SYN-ACK

Frame 3: [Sender:62319 → Receiver:5000] ACK

Screenshots of the sequence:

Sender to Receiver SYN:

No.	Time	Source	Destination	Protocol	Length	Info
11	14.006528	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
12	14.006942	127.0.0.1	127.0.0.1	UDP	44	5000 → 65139 Len=12
13	14.007459	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
14	14.007930	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
15	14.008186	127.0.0.1	127.0.0.1	TAPA	37	Tunnel - V=0, T=Type 0[Malformed Packet]
16	15.012884	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
17	15.014461	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
18	15.017740	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
19	15.019283	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
20	15.022081	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
21	15.023006	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
22	15.025126	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
23	15.025846	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
24	15.027546	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
25	15.028050	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
26	15.029783	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
27	15.030376	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
28	20.034265	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
29	20.034518	127.0.0.1	127.0.0.1	UDP	44	5000 → 65139 Len=12
30	20.034661	127.0.0.1	127.0.0.1	UDP	40	5000 → 65139 Len=8
31	20.035272	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8

Frame 11: Packet, 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface \Device\NPF_{...} id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
User Datagram Protocol, Src Port: 65139, Dst Port: 5000
Data (8 bytes)

0000 02 00 00 00 45 00 00 24 80 04 00 00 00 11 00 00E \$
0010 7f 00 00 01 7f 00 00 01 fe 73 13 88 00 10 9c 2ds
0020 fa 00 00 00 00 53 59 4eSYN

Receiver to Sender SYN-ACK:

No.	Time	Source	Destination	Protocol	Length	Info
11	14.006528	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
12	14.006942	127.0.0.1	127.0.0.1	UDP	44	5000 → 65139 Len=12
13	14.007459	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
14	14.007930	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
15	14.008186	127.0.0.1	127.0.0.1	TAPA	37	Tunnel - V=0, T=Type 0[Malformed Packet]
16	15.012884	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
17	15.014461	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
18	15.017740	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
19	15.019283	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
20	15.022081	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
21	15.023006	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
22	15.025126	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
23	15.025846	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
24	15.027546	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
25	15.028050	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
26	15.029783	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
27	15.030376	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
28	20.034265	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
29	20.034518	127.0.0.1	127.0.0.1	UDP	44	5000 → 65139 Len=12
30	20.034661	127.0.0.1	127.0.0.1	UDP	40	5000 → 65139 Len=8
31	20.035272	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8

Frame 12: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_{...} id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
User Datagram Protocol, Src Port: 5000, Dst Port: 65139
Data (12 bytes)

0000 02 00 00 00 45 00 00 28 80 05 00 00 00 11 00 00E (.....
0010 7f 00 00 01 7f 00 00 01 13 88 fe 73 00 14 2f 99s /
0020 f6 00 00 00 00 53 59 4e 2d 41 43 4bSYN -ACK

Sender to Receiver ACK:

No.	Time	Source	Destination	Protocol	Length	Info
11	14.006528	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
12	14.006942	127.0.0.1	127.0.0.1	UDP	44	5000 → 65139 Len=12
13	14.007459	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
14	14.007930	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
15	14.008186	127.0.0.1	127.0.0.1	TAP	37	Tunnel - V=0, T=Type 0[Malformed Packet]
16	15.012884	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
17	15.014461	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
18	15.017740	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
19	15.019283	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
20	15.022081	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
21	15.023086	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
22	15.025126	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
23	15.025846	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
24	15.027546	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
25	15.028050	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
26	15.029783	127.0.0.1	127.0.0.1	UDP	38	65139 → 5000 Len=6
27	15.030376	127.0.0.1	127.0.0.1	UDP	37	5000 → 65139 Len=5
28	20.034265	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8
29	20.034518	127.0.0.1	127.0.0.1	UDP	44	5000 → 65139 Len=12
30	20.034661	127.0.0.1	127.0.0.1	UDP	40	5000 → 65139 Len=8
31	20.035272	127.0.0.1	127.0.0.1	UDP	40	65139 → 5000 Len=8

Frame 13: Packet, 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface \Device\NPF_{...}, id 0	0000	02 00 00 00 45 00 00 24	80 06 00 00 00 11 00 00	E \$
Null/Loopback	0010	7f 00 00 01 7f 00 00 01	fe 73 13 88 00 10 dd 42	s B
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	cf 00 00 00 00 41 43 4b		ACK
User Datagram Protocol, Src Port: 65139, Dst Port: 5000				
Data (8 bytes)				

B) Connection Termination

Four-way teardown to terminate the connection after the session.

Sequence:

Frame N-3: [Sender:63202 → Receiver:5000] FIN

Frame N-2: [Receiver:5000 → Sender:62319] ACK

Frame N-1: [Receiver:5000 → Sender:62319] FIN

Frame N: [Sender:63202 → Receiver:5000] ACK

Screenshot of sequence:

Sender to Receiver FIN:

No.	Time	Source	Destination	Protocol	Length	Info
5	0.911317	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
6	0.911543	127.0.0.1	127.0.0.1	UDP	44	5000 → 63202 Len=12
7	0.911818	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
8	1.913524	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
9	1.913778	127.0.0.1	127.0.0.1	TAP	37	Tunnel - V=0, T=Type 0
10	3.916103	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
11	3.916421	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
12	3.916673	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
13	3.917282	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
14	3.917773	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
15	3.918173	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
34	5.921089	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
35	5.921899	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
36	5.922404	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
37	5.922737	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
38	5.923248	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
39	5.923476	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
58	20.925151	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
59	20.925744	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
60	20.925808	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
61	20.926135	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8

Frame 58: Packet, 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface \Device\NPF_{...}, id 0	0000	02 00 00 00 45 00 00 24	82 c1 00 00 00 11 00 00	E \$
Null/Loopback	0010	7f 00 00 02 7f 00 00 01	fe 42 13 88 00 10 c0 cb	
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	dd 00 00 00 00 46 49 4e		FIN
User Datagram Protocol, Src Port: 63202, Dst Port: 5000				
Data (8 bytes)				

Receiver to Sender ACK:

udpport == 5000

No.	Time	Source	Destination	Protocol	Length	Info
5	0.911317	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
6	0.911543	127.0.0.1	127.0.0.1	UDP	44	5000 → 63202 Len=12
7	0.911818	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
8	1.913524	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
9	1.913778	127.0.0.1	127.0.0.1	TAPA	37	Tunnel - V=0, T=Type 0
10	3.916103	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
11	3.916421	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
12	3.916973	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
13	3.917202	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
14	3.917773	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
15	3.918173	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
34	5.921089	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
35	5.921899	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
36	5.922404	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
37	5.922737	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
38	5.923248	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
39	5.923476	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
58	20.925516	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
59	20.925744	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
60	20.925808	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
61	20.926135	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8

Frame 59: Packet, 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface \Device\NPF_{...} id 0

Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 5000, Dst Port: 63202

Data (8 bytes)

0000 02 00 00 00 45 00 00 24 82 c2 00 00 00 11 00 00E\$.....
0010 7f 00 00 01 7f 00 00 01 13 88 f6 e2 00 10 e4 d3FIN.....
0020 cf 00 00 00 00 41 43 4bACK.....

Receiver to Sender FIN:

udpport == 5000

No.	Time	Source	Destination	Protocol	Length	Info
5	0.911317	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
6	0.911543	127.0.0.1	127.0.0.1	UDP	44	5000 → 63202 Len=12
7	0.911818	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
8	1.913524	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
9	1.913778	127.0.0.1	127.0.0.1	TAPA	37	Tunnel - V=0, T=Type 0
10	3.916103	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
11	3.916421	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
12	3.916973	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
13	3.917202	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
14	3.917773	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
15	3.918173	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
34	5.921089	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
35	5.921899	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
36	5.922404	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
37	5.922737	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
38	5.923248	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
39	5.923476	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
58	20.925516	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
59	20.925744	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
60	20.925808	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
61	20.926135	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8

Frame 60: Packet, 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface \Device\NPF_{...} id 0

Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 5000, Dst Port: 63202

Data (8 bytes)

0000 02 00 00 00 45 00 00 24 82 c2 00 00 00 11 00 00E\$.....
0010 7f 00 00 01 7f 00 00 01 13 88 f6 e2 00 10 e4 cbFIN.....
0020 dd 00 00 00 00 46 49 4aFIN.....

Sender to Receiver ACK:

udpport == 5000

No.	Time	Source	Destination	Protocol	Length	Info
5	0.911317	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
6	0.911543	127.0.0.1	127.0.0.1	UDP	44	5000 → 63202 Len=12
7	0.911818	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
8	1.913524	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
9	1.913778	127.0.0.1	127.0.0.1	TAPA	37	Tunnel - V=0, T=Type 0
10	3.916103	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
11	3.916421	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
12	3.916973	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
13	3.917202	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
14	3.917773	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
15	3.918173	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
34	5.921089	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
35	5.921899	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
36	5.922404	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
37	5.922737	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
38	5.923248	127.0.0.1	127.0.0.1	UDP	38	63202 → 5000 Len=6
39	5.923476	127.0.0.1	127.0.0.1	UDP	37	5000 → 63202 Len=5
58	20.925516	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8
59	20.925744	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
60	20.925808	127.0.0.1	127.0.0.1	UDP	40	5000 → 63202 Len=8
61	20.926135	127.0.0.1	127.0.0.1	UDP	40	63202 → 5000 Len=8

Frame 61: Packet, 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface \Device\NPF_{...} id 0

Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 63202, Dst Port: 5000

Data (8 bytes)

0000 02 00 00 00 45 00 00 24 82 c4 00 00 00 11 00 00E\$.....
0010 7f 00 00 01 7f 00 00 01 f6 e2 13 88 00 10 e4 d3ACK.....
0020 cf 00 00 00 00 41 43 4bACK.....

C) Reliable Data Transfer

ACKS and In order delivery

The top image shows a packet capture of UDP traffic on port 5000. The packet list shows frames 136 and 137. Frame 136 is a data packet from 127.0.0.1 to 127.0.0.1, length 14 bytes. Frame 137 is an ACK packet from 127.0.0.1 to 127.0.0.1, length 8 bytes. The packet details for frame 136 show the data payload as "FLOW_TEST". The packet details for frame 137 show the ACK number as 10.

The bottom image shows the same packet capture, but with frame 136 selected. The packet details for frame 136 show the data payload as "FLOW_TEST". The packet details for frame 137 show the ACK number as 10.

2 image above show frames 136–137 and illustrate in order delivery

Frame 136 is a Sender → Receiver data packet with seq = 10 (0a), ack = 0, rwnd = 0 and payload "FLOW_TEST".

Frame 137 is the matching Receiver → Sender ACK with ack = 10 and rwnd = 8, meaning the receiver has correctly received all data up to sequence 10 and still has 8 buffer slots available.

Retransmission and Loss recovery

Time	Time	Source	Destination	Protocol	Length	Info
760	578.842547	127.0.0.1	127.0.0.1	UDP	37	5000 → 5001 Len=5
761	578.844396	127.0.0.1	127.0.0.1	UDP	37	5000 → 5001 Len=5
764	578.884946	127.0.0.1	127.0.0.1	UDP	37	5000 → 5001 Len=5
554	561.285316	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
556	561.226224	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
558	561.247529	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
559	561.268663	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
561	561.289641	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
563	561.311780	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
565	561.314864	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
567	561.334362	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
569	561.355487	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
570	562.337159	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
572	562.337555	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
573	562.337915	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
574	562.338228	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
575	562.338469	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
578	562.341481	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
581	562.362847	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
587	562.406819	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10

> Frame 569: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{...} Loopback, id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> User Datagram Protocol, Src Port: 5001, Dst Port: 5000

> Data (10 bytes)

Data: 84070000004d53475f37

[Length: 10]

0000 02 00 00 00 45 00 00 26 74 aa 00 00 00 11 00 00E..&.....
0010 7f 00 00 01 7f 00 00 01 13 89 13 88 00 12 a3 e3
0020 84 07 00 00 00 4d 53 47 5f 37MSG_7

573	562.337915	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
574	562.338228	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
575	562.338469	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
578	562.341481	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
581	562.362847	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10
587	562.406819	127.0.0.1	127.0.0.1	UDP	42	5001 → 5000 Len=10

> Frame 575: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{...} Loopback, id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> User Datagram Protocol, Src Port: 5001, Dst Port: 5000

> Data (10 bytes)

Data: 84070000004d53475f37

[Length: 10]

0000 02 00 00 00 45 00 00 26 74 b0 00 00 00 11 00 00E..&.....
0010 7f 00 00 01 7f 00 00 01 13 89 13 88 00 12 a3 e3
0020 84 07 00 00 00 4d 53 47 5f 37MSG_7

Packet MSG_7 exists in both Frame 569 and 575, indicating that the packet was initially lost but retransmitted.

D) State Maintenance

The `Packet.encode()` format in `packet.py` are (checksum, seq,ack,rwnd,payload)
Sequence Number

```

29 12.847101 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
30 12.847986 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
31 12.850662 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
32 13.853529 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
33 13.854269 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
34 13.855796 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
35 14.858666 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
36 14.859584 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
37 15.860156 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
38 15.860822 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
39 15.861001 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
40 15.861913 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
41 15.863070 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
42 15.864724 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
43 15.873219 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
44 15.874688 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
45 15.883583 127.0.0.1 127.0.0.1 UDP 38 55932 > 5000 Len=6
46 15.884812 127.0.0.1 127.0.0.1 UDP 37 5000 > 55932 Len=5
47 15.895940 127.0.0.1 127.0.0.1 UDP 40 55932 > 5000 Len=8
48 15.896392 127.0.0.1 127.0.0.1 UDP 40 5000 > 55932 Len=8
49 15.896558 127.0.0.1 127.0.0.1 UDP 40 5000 > 55932 Len=8
> Frame 38: 38 bytes on wire (304 bits), 38 bytes captured (304 bits) on interface Device\VF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 55932, Dst Port: 5000
Data (6 bytes)
[Length: 6]
0000  02 00 00 00 45 00 00 22 77 0c 00 00 00 11 00 00  ....E..w.....
0010  7f 00 00 01 7f 00 00 01 da 7c 13 88 00 0e cc 83  ....|.....
0020  47 83 00 00 00 4c  ....L

```

ACKs number

199 13.209142 127.0.0.1 127.0.0.1 UDP 37 5000 → 51575 Len=5

200 13.217079 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

201 14.208865 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

202 14.209317 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

203 14.209771 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

204 14.210163 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

205 14.211561 127.0.0.1 127.0.0.1 UDP 37 5000 → 51575 Len=5

206 14.212163 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

207 14.213462 127.0.0.1 127.0.0.1 UDP 37 5000 → 51575 Len=5

208 14.214850 127.0.0.1 127.0.0.1 UDP 37 5000 → 51575 Len=5

209 14.216349 127.0.0.1 127.0.0.1 UDP 37 5000 → 51575 Len=5

210 14.216861 127.0.0.1 127.0.0.1 UDP 137 51575 → 5000 Len=105

> Frame 209: 37 bytes on wire (296 bits), 37 bytes captured (296 bits) on interface \Device\NPF_{...}_Loopback, id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> User Datagram Protocol, Src Port: 5000, Dst Port: 51575

> Data (5 bytes)

Data: 26001e0000

[Length: 5]

The ACK number here is in correct order (1d -> 1e from frame 208 to frame 209).

Sender/Receiver window

19 16.128022 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

20 16.137772 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

21 16.148464 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

22 16.149789 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

23 16.158513 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

24 17.150757 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

25 17.151251 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

26 17.151907 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

27 17.151908 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

28 17.155218 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

33 18.157327 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

34 18.157880 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

35 18.158757 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

36 18.159180 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

37 18.160049 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

38 18.161886 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

39 18.170454 127.0.0.1 127.0.0.1 UDP 40 65088 → 5000 Len=8

40 18.171531 127.0.0.1 127.0.0.1 UDP 40 5000 → 65088 Len=8

41 18.171557 127.0.0.1 127.0.0.1 UDP 40 5000 → 65088 Len=8

42 18.171869 127.0.0.1 127.0.0.1 UDP 40 65088 → 5000 Len=8

> Frame 26: 37 bytes on wire (296 bits), 37 bytes captured (296 bits) on interface \Device\NPF_{...}_Loopback, id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> User Datagram Protocol, Src Port: 5000, Dst Port: 65088

> Data (5 bytes)

Data: 000030000

[Length: 5]

19 16.128022 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

20 16.137772 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

21 16.148464 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

22 16.149789 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

23 16.158513 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

24 17.150757 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

25 17.151251 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

26 17.151907 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

27 17.151908 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

28 17.155218 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

33 18.157327 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

34 18.157880 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

35 18.158757 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

36 18.159180 127.0.0.1 127.0.0.1 UDP 38 65088 → 5000 Len=6

37 18.160049 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

38 18.161886 127.0.0.1 127.0.0.1 UDP 37 5000 → 65088 Len=5

39 18.170454 127.0.0.1 127.0.0.1 UDP 40 65088 → 5000 Len=8

40 18.171531 127.0.0.1 127.0.0.1 UDP 40 5000 → 65088 Len=8

41 18.171557 127.0.0.1 127.0.0.1 UDP 40 5000 → 65088 Len=8

42 18.171869 127.0.0.1 127.0.0.1 UDP 40 65088 → 5000 Len=8

> Frame 28: 37 bytes on wire (296 bits), 37 bytes captured (296 bits) on interface \Device\NPF_{...}_Loopback, id 0

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> User Datagram Protocol, Src Port: 5000, Dst Port: 65088

> Data (5 bytes)

Data: 000030000

[Length: 5]

The rwnd is consecutively 8, which is correct as we set the buffer as 8 for our implementation

2. Performance

To test the performance of our protocol, we implemented a performance test suite (can be found in the code as `prove_performance.py`). The script will validate that the protocol has adequate performance by running 5 separate tests.

1. Reliability Test:

- What it test: 100% data delivery despite 30% simulated packet loss
- How: Sends 20 sequential messages, verifies all are acknowledged
- Success criteria: `sender.base >= number_of_packets_sent`
- Code snippet:

2. Throughput Test:

- What it test: Data transfer rate under realistic conditions
- How: Sends 100 packets (100 bytes each), measures time elapsed
- Success criteria: > 10 KB/s with 30% packet loss

3. Congestion Control Test:

- What it test: CWND adapts to network conditions
- How: Sends 20 packets, tracks CWND evolution
- Success criteria: CWND grows from initial value (demonstrates slow start/congestion avoidance)

4. Connection management test:

- What it test: Proper handshake and teardown
- How: Performs `connect()`, verifies state = ESTABLISHED, calls `close()`, verifies state = CLOSED
- Success criteria: Both handshake and teardown complete successfully

5. Flow Control:

- What it test: Sender respects receiver's advertised window
- How: Sends 15 packets, monitors RWND values from ACKs
- Success criteria: Sender tracks and uses RWND in window calculations

Result from script:

=====	
FINAL SUMMARY	
=====	
1. Reliability:	PROVEN
2. Throughput:	ACCEPTABLE
3. Congestion Control:	PROVEN
4. Flow Control:	PROVEN
5. Connection Mgmt:	PROVEN
=====	

3. Flow control

We used the same script that tested sending 15 packets to the receiver using the [sender.py](#) class, monitoring the rwnd in the ACKs packet and observing if sender tracks and uses rwnd value in calculations.

Result from script:

```
=====
PROOF 4: FLOW CONTROL - Respects Receiver Window
=====
```

```
Sender: initiating handshake...
Sender: SENT SYN
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: RECEIVED SYN-ACK
Sender: SENT ACK - connection established
```

```
Tracking receiver window (RWND) from ACKs...
```

```
Sender: sent seq=0, cwnd=1.00, ssthresh=8
Sender: got ACK=0, rwnd=8, cwnd=1.00
  Packet 1: RWND = 8, Effective Window = 2
DROPPING PACKET ON PURPOSE
Sender: sent seq=1, cwnd=2.00, ssthresh=8
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1
  Resending seq=1
DROPPING PACKET ON PURPOSE
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1
  Resending seq=1
Sender: got ACK=1, rwnd=8, cwnd=1.00
  Packet 2: RWND = 8, Effective Window = 2
Sender: sent seq=2, cwnd=2.00, ssthresh=1
Sender: got ACK=2, rwnd=8, cwnd=2.00
  Packet 3: RWND = 8, Effective Window = 2
Sender: sent seq=3, cwnd=2.50, ssthresh=1
Sender: got ACK=3, rwnd=8, cwnd=2.50
  Packet 4: RWND = 8, Effective Window = 2
Sender: sent seq=4, cwnd=2.90, ssthresh=1
Sender: got ACK=4, rwnd=8, cwnd=2.90
  Packet 5: RWND = 8, Effective Window = 3
```

```
Sender: sent seq=5, cwnd=3.24, ssthresh=1
Sender: got ACK=5, rwnd=8, cwnd=3.24
  Packet 6: RWND = 8, Effective Window = 3
Sender: sent seq=6, cwnd=3.55, ssthresh=1
Sender: got ACK=6, rwnd=8, cwnd=3.55
Sender: sent seq=7, cwnd=3.83, ssthresh=1
Sender: got ACK=7, rwnd=8, cwnd=3.83
Sender: sent seq=8, cwnd=4.10, ssthresh=1
Sender: got ACK=8, rwnd=8, cwnd=4.10
DROPPING PACKET ON PURPOSE
Sender: sent seq=9, cwnd=4.34, ssthresh=1
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=2.1697198463621725
  Resending seq=9
Sender: got ACK=9, rwnd=8, cwnd=1.00
Sender: sent seq=10, cwnd=2.00, ssthresh=2.1697198463621725
Sender: got ACK=10, rwnd=8, cwnd=2.00
  Packet 11: RWND = 8, Effective Window = 3
DROPPING PACKET ON PURPOSE
Sender: sent seq=11, cwnd=3.00, ssthresh=2.1697198463621725
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1.5
  Resending seq=11
Sender: got ACK=11, rwnd=8, cwnd=1.00
Sender: sent seq=12, cwnd=2.00, ssthresh=1.5
Sender: got ACK=12, rwnd=8, cwnd=2.00
Sender: sent seq=13, cwnd=2.50, ssthresh=1.5
Sender: got ACK=13, rwnd=8, cwnd=2.50
DROPPING PACKET ON PURPOSE
Sender: sent seq=14, cwnd=2.90, ssthresh=1.5
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1.45
  Resending seq=14
Sender: got ACK=14, rwnd=8, cwnd=1.00
Sender: Initiating connection close (FIN)...
Sender: SENT FIN
Sender: RECEIVED ACK for FIN
Sender: RECEIVED FIN from receiver
```

```
Sender: SENT final ACK  
Sender: Entering TIME_WAIT state (2 seconds)...  
Sender: Connection closed
```

RWND statistics:

```
Min RWND: 8  
Max RWND: 8  
Avg RWND: 8.0
```

PROOF: Sender tracks and respects receiver window

=====

Congestion control

For congestion control, we follow the same code of the test suite. We check if the cwnd of the sender adapts to the network conditions. The script will send 20 packets and track the cwnd value throughout. If the cwnd value grows from initial value and demonstrates the slow start, congestion avoidance it'll prove that the protocol implemented a congestion control mechanism.

Screenshot from script:

PROOF 3: CONGESTION CONTROL - CWND Adapts to Network

```
=====
Sender: initiating handshake...
Sender: SENT SYN
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: Handshake timeout, retrying...
Sender: RECEIVED SYN-ACK
Sender: SENT ACK – connection established
```

Initial state:

```
CWND:      1
SSTHRESH:  8
```

Sending packets and tracking CWND...

```
Sender: sent seq=0, cwnd=1.00, ssthresh=8
Sender: got ACK=0, rwnd=8, cwnd=1.00
  Packet 1: CWND = 2.00, SSTHRESH = 8
Sender: sent seq=1, cwnd=2.00, ssthresh=8
Sender: got ACK=1, rwnd=8, cwnd=2.00
  Packet 2: CWND = 3.00, SSTHRESH = 8
Sender: sent seq=2, cwnd=3.00, ssthresh=8
Sender: got ACK=2, rwnd=8, cwnd=3.00
  Packet 3: CWND = 4.00, SSTHRESH = 8
DROPPING PACKET ON PURPOSE
Sender: sent seq=3, cwnd=4.00, ssthresh=8
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=2.0
  Resending seq=3
Sender: got ACK=3, rwnd=8, cwnd=1.00
  Packet 4: CWND = 2.00, SSTHRESH = 2.0
Sender: sent seq=4, cwnd=2.00, ssthresh=2.0
Sender: got ACK=4, rwnd=8, cwnd=2.00
  Packet 5: CWND = 2.50, SSTHRESH = 2.0
Sender: sent seq=5, cwnd=2.50, ssthresh=2.0
Sender: got ACK=5, rwnd=8, cwnd=2.50
  Packet 6: CWND = 2.90, SSTHRESH = 2.0
```

```
Sender: sent seq=6, cwnd=2.90, ssthresh=2.0
Sender: got ACK=6, rwnd=8, cwnd=2.90
  Packet 7: CWND = 3.24, SSTHRESH = 2.0
Sender: sent seq=7, cwnd=3.24, ssthresh=2.0
Sender: got ACK=7, rwnd=8, cwnd=3.24
  Packet 8: CWND = 3.55, SSTHRESH = 2.0
Sender: sent seq=8, cwnd=3.55, ssthresh=2.0
Sender: got ACK=8, rwnd=8, cwnd=3.55
  Packet 9: CWND = 3.83, SSTHRESH = 2.0
Sender: sent seq=9, cwnd=3.83, ssthresh=2.0
Sender: got ACK=9, rwnd=8, cwnd=3.83
  Packet 10: CWND = 4.10, SSTHRESH = 2.0
DROPPING PACKET ON PURPOSE
Sender: sent seq=10, cwnd=4.10, ssthresh=2.0
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=2.0476273161293896
  Resending seq=10
Sender: got ACK=10, rwnd=8, cwnd=1.00
  Packet 11: CWND = 2.00, SSTHRESH = 2.0476273161293896
Sender: sent seq=11, cwnd=2.00, ssthresh=2.0476273161293896
Sender: got ACK=11, rwnd=8, cwnd=2.00
Sender: sent seq=12, cwnd=3.00, ssthresh=2.0476273161293896
Sender: got ACK=12, rwnd=8, cwnd=3.00
Sender: sent seq=13, cwnd=3.33, ssthresh=2.0476273161293896
Sender: got ACK=13, rwnd=8, cwnd=3.33
DROPPING PACKET ON PURPOSE
Sender: sent seq=14, cwnd=3.63, ssthresh=2.0476273161293896
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1.8166666666666667
  Resending seq=14
DROPPING PACKET ON PURPOSE
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1
  Resending seq=14
DROPPING PACKET ON PURPOSE
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
  NEW cwnd=1, NEW ssthresh=1
  Resending seq=14
Sender: got ACK=14, rwnd=8, cwnd=1.00
```



```

DROPPING PACKET ON PURPOSE
Sender: sent seq=15, cwnd=2.00, ssthresh=1
Sender: TIMEOUT → RESEND WINDOW (CONGESTION)
      NEW cwnd=1, NEW ssthresh=1
      Resending seq=15
Sender: got ACK=15, rwnd=8, cwnd=1.00
      Packet 16: CWND = 2.00, SSTHRESH = 1
Sender: sent seq=16, cwnd=2.00, ssthresh=1
Sender: got ACK=16, rwnd=8, cwnd=2.00
Sender: sent seq=17, cwnd=2.50, ssthresh=1
Sender: got ACK=17, rwnd=8, cwnd=2.50
Sender: sent seq=18, cwnd=2.90, ssthresh=1
Sender: got ACK=18, rwnd=8, cwnd=2.90
Sender: sent seq=19, cwnd=3.24, ssthresh=1
Sender: got ACK=19, rwnd=8, cwnd=3.24

Final state:
      CWND:      3.55
      SSTHRESH:  1
      Max CWND:  4.10
Sender: Initiating connection close (FIN)...
Sender: SENT FIN
Sender: RECEIVED ACK for FIN
Sender: RECEIVED FIN from receiver
Sender: SENT final ACK
Sender: Entering TIME_WAIT state (2 seconds)...
Sender: Connection closed

PROOF: CWND shows adaptive behavior (grew from 1 to 4.10)

```

III. Test with other traffic (Prove Fairness)

We ran the simple test between receiver side and sender side over the network by sitting together and joining the same LAN. The receiver side will check its IPv4 address, enable firewall rules for port 5000 for UDP to receive network traffic. The sender side then sends the packets to port 5000 on the specified IP.

Wireshark screenshot of the over the internet on receiver side:

udp.port == 5000					
No.	Time	Source	Destination	Protocol	Length Info
716	70.958315	10.63.4.78	10.63.4.69	UDP	50 59649 → 5000 Len=8
717	70.958546	10.63.4.69	10.63.4.78	UDP	54 5000 → 59649 Len=12
718	70.984417	10.63.4.78	10.63.4.69	UDP	50 59649 → 5000 Len=8
719	70.988098	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
720	70.989782	10.63.4.69	10.63.4.78	TAPA	47 Tunnel - V=0, T=Type 0
721	71.001907	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
722	71.002527	10.63.4.69	10.63.4.78	UDP	47 5000 → 59649 Len=5
723	71.011397	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
724	71.012739	10.63.4.69	10.63.4.78	UDP	47 5000 → 59649 Len=5
725	71.020123	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
726	71.021197	10.63.4.69	10.63.4.78	UDP	47 5000 → 59649 Len=5
727	71.032126	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
728	71.033370	10.63.4.69	10.63.4.78	UDP	47 5000 → 59649 Len=5
729	71.043173	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
730	71.054388	10.63.4.78	10.63.4.69	UDP	48 59649 → 5000 Len=6
731	71.056074	10.63.4.69	10.63.4.78	UDP	47 5000 → 59649 Len=5
732	71.064253	10.63.4.78	10.63.4.69	UDP	50 59649 → 5000 Len=8
733	71.065084	10.63.4.69	10.63.4.78	UDP	50 5000 → 59649 Len=8
734	71.065138	10.63.4.69	10.63.4.78	UDP	50 5000 → 59649 Len=8
735	71.072027	10.63.4.78	10.63.4.69	UDP	50 59649 → 5000 Len=8

Wireshark screenshot of the over the internet on sender side:

No.	Time	Source	Destination	Protoc	Leng	Info
869	32.015401	10.63.4.78	10.63.4.69	UDP	50	59649 → 5000 Len=8
875	32.156736	10.63.4.69	10.63.4.78	UDP	54	5000 → 59649 Len=12
876	32.157293	10.63.4.78	10.63.4.69	UDP	50	59649 → 5000 Len=8
877	32.157811	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6
879	32.188304	10.63.4.69	10.63.4.78	TAPA	47	Tunnel - V=0, T=Type 0
880	32.188948	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6
881	32.198876	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6
882	32.201697	10.63.4.69	10.63.4.78	UDP	47	5000 → 59649 Len=5
883	32.211121	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6
884	32.211239	10.63.4.69	10.63.4.78	UDP	47	5000 → 59649 Len=5
885	32.218433	10.63.4.69	10.63.4.78	UDP	47	5000 → 59649 Len=5
886	32.222277	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6
887	32.231203	10.63.4.69	10.63.4.78	UDP	47	5000 → 59649 Len=5
888	32.233292	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6
889	32.243466	10.63.4.78	10.63.4.69	UDP	48	59649 → 5000 Len=6

To see these in detail, you can check our two pcap files included in the zip to see the correct protocol behaviour being transmitted from two different IPv4 addresses (over the LAN). One is the receiver side pcap and the other is the sender side pcap.

This is tested by setting up two computers: one acting as the receiver and the other acting as the sender

Fairness proof:

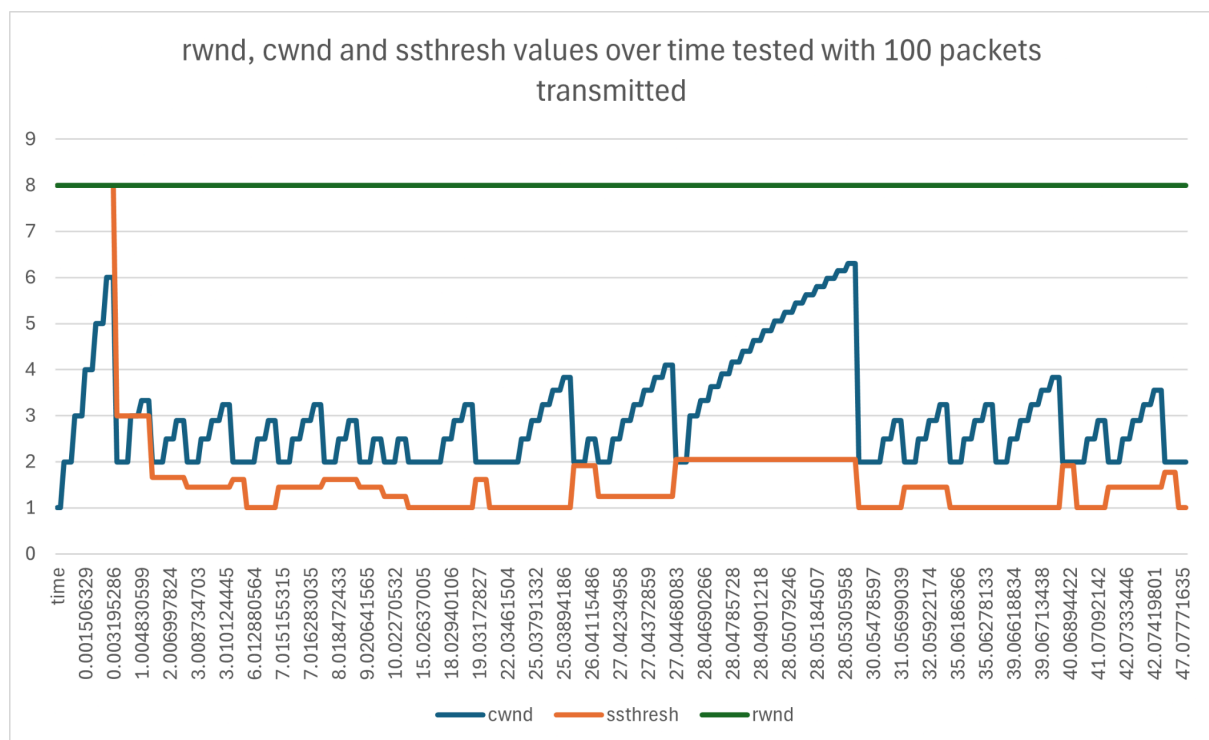
Our protocol implemented TCP-like AIMD which follows TCP Reno's behaviour. The data collected via tests throughout this report is sufficient to show that the protocol indeed follows TCP Reno's behaviour. Since TCP Reno is proven fair by design, our protocol hence is also

fair as it follows the same behaviour. Additionally, we observe the cwnd behaviour. According to the graph in the below section (test run detailed data), our protocol produces the same sawtooth pattern like TCP in a controlled test where packets are dropped artificially.

Hence, in conclusion, since our protocol implemented AIMD similar to TCP, it is considered fair.

IV. Test run detailed data

We ran the Receiver and Sender script with a controlled test sending 100 packets within each other and documented the cwnd, rwnd, ssthresh and other connection metrics in an excel file that can be found in the submitted zip file. Below is the chart showing the 3 main metrics that proves the flow and congestion control of the protocol:



The X axis stands for time in seconds, and the Y axis is the value of the variables.

References:

- [1] D. Chiu and R. Jain, "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," Computer Networks and ISDN Systems, 1989.
- [2] S. Floyd and K. Fall, "Promoting the Use of End-to-End Congestion Control in the Internet," IEEE/ACM Transactions

on Networking, 1999.

[3] V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM
Computer Communication Review, 1988.